Aula — Conceitos

Neurônio Artificial e Funções de Ativação

Objetivos desta sessão

- Descrever o neurônio biológico, a analogia com o neurônio artificial e os limites dessa analogia.
- Calcular a saída de um neurônio: soma ponderada, viés e função de ativação.
- Interpretar o neurônio como separador linear e reconhecer o que ele não consegue fazer sozinho (o problema do XOR).
- Explicar por que, sem uma função de ativação não linear, uma rede com várias camadas equivale a uma única camada.
- Conhecer degrau, sigmoide, tangente hiperbólica, ReLU, Leaky ReLU e softmax: fórmula, faixa de valores e derivada.
- Reconhecer a saturação e o gradiente que desaparece, e escolher a ativação adequada para cada posição da rede.

1. Onde estamos e o que muda hoje

Até aqui trabalhamos com modelos clássicos: regressão, classificação e agrupamento, sempre com o ciclo preparar os dados, validar e avaliar. Na introdução às redes neurais artificiais vimos o histórico e a motivação do campo. Hoje abrimos a caixa e olhamos para a **unidade básica** de toda rede neural: o neurônio artificial.

Uma ideia vai nos acompanhar na aula inteira: você já conhece neurônios. A regressão linear e a regressão logística são, cada uma, um único neurônio com uma ativação diferente (Seção 3.1). O que as redes neurais acrescentam é empilhar muitos deles, e isso só funciona por causa da função de ativação (Seção 4).

1.1. Do neurônio biológico ao artificial

Neurônio biológico	Neurônio artificial	Papel
Dendritos	Entradas x_i	recebem os sinais de fora
Força de cada sinapse	Pesos w_i	dizem o quanto cada entrada importa
Corpo celular somando os estímulos	Soma ponderada z	combina todas as entradas em um número
Limiar de disparo	Viés b e função de ativação φ	decidem se e quanto o neurônio "dispara"
Axônio	Saída y	envia o resultado adiante

> **A analogia é inspiração, não modelo fiel**
> O neurônio real é um sistema dinâmico, com dinâmica temporal e química. O neurônio artificial é apenas uma **função matemática** de entradas numéricas. Use a analogia para memorizar a estrutura, e não para tirar conclusões sobre o cérebro.

2. O modelo matemático

Um neurônio com n entradas calcula em duas etapas. Primeiro, a **soma ponderada** das entradas mais um termo constante, o viés. Depois, aplica a **função de ativação** φ ao resultado:

```
z = w₁x₁ + w₂x₂ + ... + wₙxₙ + b = w · x + b
y = φ(z)
```

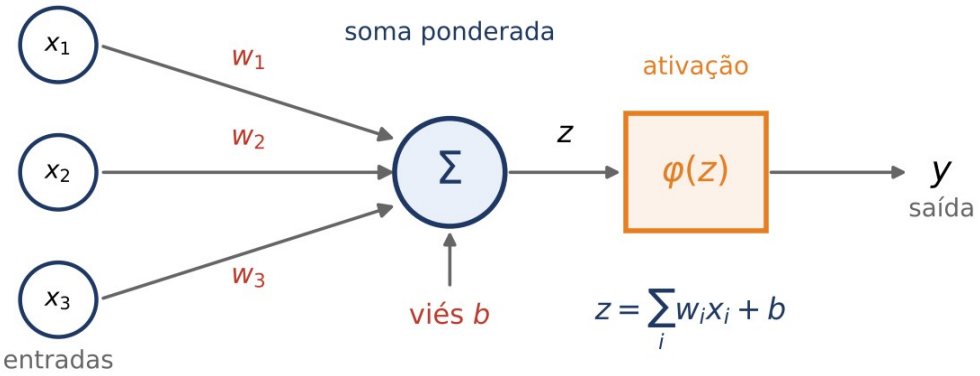


Figura 1 — o neurônio artificial: entradas, pesos, soma ponderada com viés, função de ativação e saída.

Os **pesos** w e o **viés** b são os parâmetros que o treinamento vai aprender. A função de ativação e o número de entradas são escolhas do projetista (hiperparâmetros). Em NumPy o cálculo inteiro cabe em uma linha:

```
z = np.dot(w, x) + b    # soma ponderada + viés
y = phi(z)    # phi é a função de ativação escolhida
```

2.1. Exemplo feito à mão

Um neurônio com três entradas x = (2; −1; 3), pesos w = (0,5; 1,0; −0,2) e viés b = 0,3.

```
z = 0,5·2 + 1,0·(−1) + (−0,2)·3 + 0,3 = 1,0 − 1,0 − 0,6 + 0,3 = −0,3
```

A mesma soma z = −0,3 produz saídas muito diferentes conforme a ativação:

Ativação	Regra	Saída para z = −0,3
Degrau	1 se z ≥ 0; senão 0	0
Sigmoide	$1 / (1 + e^{-z})$	0,426
Tangente hiperbólica	tanh(z)	−0,291
ReLU	máx(0, z)	0
Leaky ReLU (α = 0,01)	z se z > 0; senão 0,01·z	−0,003

2.2. O papel do viés

Sem o viés, a fronteira de decisão $w \cdot x = 0$ passaria sempre pela origem. O viés **desloca** a fronteira. Com a ativação em degrau, o neurônio dispara quando $w \cdot x \geq -b$, ou seja, **$-b$ funciona como o limiar** de disparo. Um viés muito negativo exige entradas fortes para disparar; um viés positivo faz o neurônio disparar com facilidade.

3. O neurônio como separador linear

Com a ativação em degrau, o neurônio responde 1 quando $w \cdot x + b \geq 0$ e 0 caso contrário. A equação $w \cdot x + b = 0$ descreve uma **reta** (em duas dimensões) ou um hiperplano (em mais dimensões): o neurônio divide o espaço em dois lados.

Portas lógicas AND e OR

Com duas entradas binárias, basta escolher w e b adequados. Para AND usamos $w = (1; 1)$ e $b = -1{,}5$; para OR, $w = (1; 1)$ e $b = -0{,}5$:

x_1	x_2	z (AND)	y (AND)	z (OR)	y (OR)
0	0	−1,5	0	−0,5	0
0	1	−0,5	0	0,5	1
1	0	−0,5	0	0,5	1
1	1	0,5	1	1,5	1

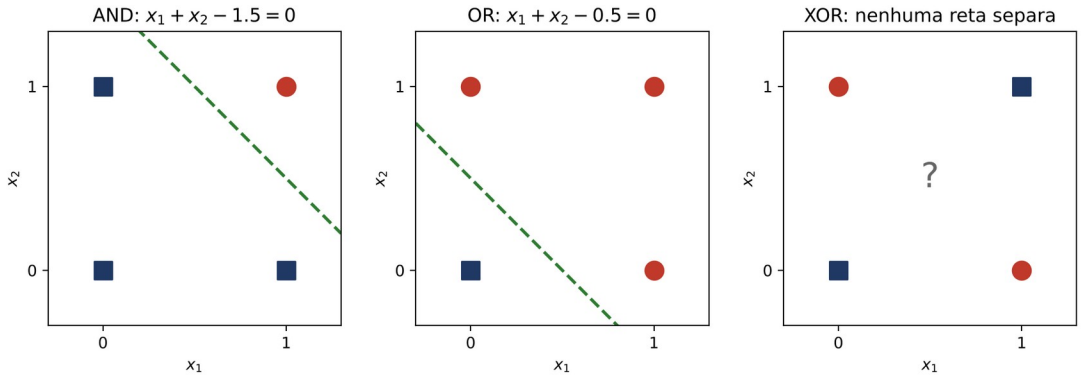


Figura 2 — AND e OR são separáveis por uma reta; o XOR (vale 1 quando as entradas diferem) não é.

O XOR é o exemplo clássico do **limite do neurônio isolado**: nenhuma reta separa os pontos de saída 1 dos de saída 0. A saída para esse limite são as camadas ocultas, que veremos nas próximas sessões, e elas só funcionam com a ajuda da função de ativação (Seção 4).

3.1. Você já conhece neurônios

Modelo que já vimos	É um neurônio com…	Função de perda usual
Regressão linear	ativação identidade: y = z	erro quadrático
Regressão logística	ativação sigmoide: y = σ(z)	entropia cruzada
Perceptron (veremos adiante)	ativação degrau	regra de correção de erro

4. Por que precisamos da função de ativação

Suponha uma rede de duas camadas em que cada neurônio **só faz a soma ponderada** (ativação identidade). Com W_1, b_1 na primeira camada e W_2, b_2 na segunda:

```
h = W₁x + b₁
y = W₂h + b₂ = (W₂W₁)x + (W₂b₁ + b₂) = W'x + b'
```

O resultado é de novo uma função linear de x: **duas camadas lineares equivalem a uma**. Isso vale para 3, 10 ou 100 camadas. Um exemplo numérico com uma única entrada: se h = 2x − 1 e y = −3h + 4, então y = −3(2x − 1) + 4 = **−6x + 7**. A pilha inteira é só uma reta.

> **A não linearidade é o que dá poder à rede**
>
> A função de ativação entre as camadas quebra essa equivalência. Com h = ReLU(2x − 1), por exemplo, a saída deixa de ser uma reta e passa a ter um "cotovelo". Combinando muitos cotovelos, a rede aproxima curvas complicadas e resolve problemas como o XOR. **Sem ativação não linear, profundidade não acrescenta nada.**

5. As principais funções de ativação

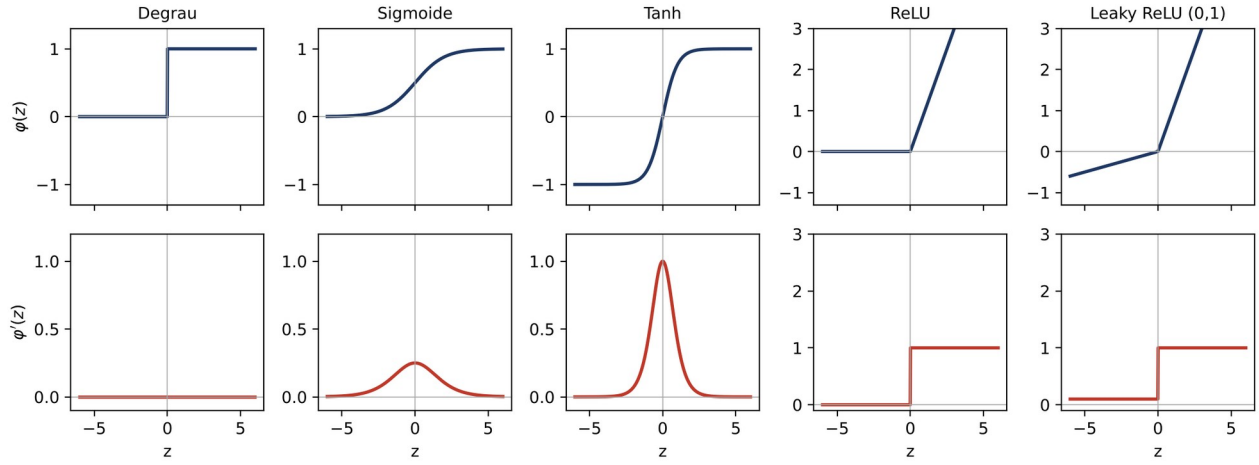


Figura 3 — funções de ativação (linha de cima) e suas derivadas (linha de baixo). No Leaky ReLU o gráfico usa α = 0,1 só para tornar a inclinação visível.

5.1. Degrau

```
φ(z) = 1 se z ≥ 0; φ(z) = 0 caso contrário
```

Faixa {0; 1}. É o neurônio original (McCulloch-Pitts e perceptron). Tem um defeito grave para o treinamento moderno: a derivada é zero em todo ponto (exceto na origem, onde não existe), então o gradiente não tem como orientar o ajuste dos pesos.

5.2. Sigmoide (logística)

```
σ(z) = 1 / (1 + e⁻ᶻ)     # faixa (0, 1)
σ'(z) = σ(z) · (1 − σ(z))     # máximo 0,25 em z = 0
```

Suave e diferenciável; comprime qualquer número em (0, 1), o que permite ler a saída como uma probabilidade. É a ativação da regressão logística e a escolha clássica para a **saída de classificação binária**. Tem dois problemas: não é centrada em zero e **satura** nas pontas (derivada próxima de zero), o que prejudica o treinamento de redes profundas (Seção 6).

5.3. Tangente hiperbólica (tanh)

```
tanh(z) = (eᶻ − e⁻ᶻ) / (eᶻ + e⁻ᶻ)    # faixa (−1, 1)
tanh'(z) = 1 − tanh²(z)    # máximo 1 em z = 0
```

Parecida com a sigmoide, mas **centrada em zero** e com derivada máxima 1 (quatro vezes a da sigmoide). Costuma treinar melhor que a sigmoide nas camadas ocultas, embora também sature. Ainda aparece em arquiteturas para sequências.

5.4. ReLU (Rectified Linear Unit)

```
ReLU(z) = máx(0, z)    # faixa [0, ∞)
ReLU'(z) = 1 se z > 0; 0 se z < 0
```

Barata de calcular e **não satura** para z positivo: a derivada vale 1 e o gradiente passa sem ser atenuado. Zera as entradas negativas, o que deixa parte dos neurônios inativos (ativação esparsa). É a escolha padrão para as camadas ocultas. O risco é a **ReLU morta**: um neurônio cujo z fica sempre negativo tem derivada 0 e deixa de aprender.

5.5. Leaky ReLU

```
LeakyReLU(z) = z se z > 0; α·z caso contrário    # α pequeno, tipicamente 0,01
```

Mantém uma inclinação pequena para z negativo, de modo que a derivada nunca é exatamente zero. É a alternativa quando muitos neurônios morrem com a ReLU.

5.6. Softmax (camada de saída multiclasse)

```
softmax(z)ₖ = exp(zₖ) / Σⱼ exp(zⱼ)    # soma das saídas = 1
```

Diferente das anteriores, a softmax não age em um número isolado: recebe o vetor de pontuações da camada de saída e o transforma em uma **distribuição de probabilidade**. Exemplo com z = (2; 1; 0,1): os exponenciais são 7,389; 2,718 e 1,105, somando 11,213; as probabilidades são **0,659; 0,242; 0,099**. A pontuação maior recebe a probabilidade maior e as três somam 1.

5.7. Resumo e valores de referência

Função	Faixa	Derivada máx.	Uso típico	Cuidado
Degrau	{0; 1}	0 (quase sempre)	histórico; perceptron	não treina por gradiente
Sigmoide	(0; 1)	0,25	saída binária	satura; não centrada
Tanh	(−1; 1)	1	ocultas (em alguns casos)	satura nas pontas
ReLU	[0; ∞)	1	camadas ocultas (padrão)	neurônios mortos
Leaky ReLU	(−∞; ∞)	1	ocultas, se a ReLU morre	escolher α
Softmax	(0; 1), soma 1	—	saída multiclasse	só na saída

Valores calculados para conferir à mão ou no laboratório:

z	σ(z)	σ'(z)	tanh(z)	tanh'(z)	ReLU(z)
−4	0,018	0,018	−0,999	0,001	0
−2	0,119	0,105	−0,964	0,071	0
−1	0,269	0,197	−0,762	0,420	0

z	σ(z)	σ'(z)	tanh(z)	tanh'(z)	ReLU(z)
0	0,500	0,250	0,000	1,000	0
1	0,731	0,197	0,762	0,420	1
2	0,881	0,105	0,964	0,071	2
4	0,982	0,018	0,999	0,001	4

6. Saturação e gradiente que desaparece

O treinamento de uma rede ajusta os pesos usando as **derivadas** das ativações; o algoritmo para isso, a retropropagação, será visto adiante. O que importa agora é que o gradiente que chega às primeiras camadas é um **produto** de derivadas, uma por camada.

- A sigmoide tem derivada no máximo 0,25 (em $z = 0$) e quase zero nas pontas: $\sigma'(\pm 4) = 0{,}018$. Em uma cadeia de 5 camadas, o produto é no máximo $0{,}25^5 \approx 0{,}001$; com 10 camadas, $0{,}25^{10} \approx 10^{-6}$. O gradiente praticamente **desaparece** e as primeiras camadas quase não aprendem.
- A tanh melhora (derivada máxima 1), mas também satura para $|z|$ grande.
- A ReLU tem derivada 1 para $z > 0$, então o gradiente atravessa as camadas sem ser atenuado. O preço é a ReLU morta, mitigada pela Leaky ReLU.

> **Ligação com a preparação dos dados**
>
> Entradas padronizadas mantêm z perto de zero no início do treinamento, justamente a região em que sigmoide e tanh têm derivada grande. Padronizar não é só ajuda para distâncias: é também condição para a rede treinar bem.

7. Como escolher a ativação

Onde / para quê	Ativação	Por quê
Camadas ocultas (ponto de partida)	ReLU	simples, não satura para z positivo, treina rápido
Camadas ocultas, se há muitos neurônios mortos	Leaky ReLU	mantém gradiente para z negativo
Saída de regressão (valor real)	identidade (nenhuma)	a saída precisa cobrir qualquer valor real
Saída de classificação binária (um neurônio)	sigmoide	saída entre 0 e 1, lida como probabilidade
Saída multiclasse com classes excludentes	softmax	probabilidades que somam 1
Saída multirrótulo (várias classes ao mesmo tempo)	sigmoide em cada saída	cada rótulo é decidido de forma independente

Erros frequentes

- Usar sigmoide em todas as camadas ocultas de uma rede profunda (gradiente desaparece).
- Esquecer o viés e forçar toda fronteira a passar pela origem.
- Usar degrau e esperar que o treinamento por gradiente funcione.
- Colocar softmax nas camadas ocultas: ela serve para a saída.
- Aplicar ReLU na saída de uma regressão que pode produzir valores negativos.

8. No laboratório

No laboratório que acompanha esta aula você vai implementar o neurônio artificial em NumPy, comparar as funções de ativação sobre o mesmo conjunto de entradas, reproduzir as portas AND e OR com pesos escolhidos à mão e observar a saturação da sigmoide em números.

9. Exercícios resolvidos

Cinco exercícios resolvidos passo a passo, cada um com a ideia-chave ao final. Estude-os antes de resolver os exercícios da Seção 10: eles usam os mesmos raciocínios com outros números.

Exercício 1 — Calcular a saída de um neurônio

Um neurônio tem entradas x = (1; −2), pesos w = (0,8; 0,5) e viés b = 0,4. (a) Calcule z. (b) Calcule a saída com as ativações degrau, sigmoide, tanh e ReLU. (c) O que acontece com z e com a saída do degrau se o viés passar a ser b = −0,4?

Resolução.

(a) O primeiro passo é sempre a soma ponderada: multiplicamos cada entrada pelo seu peso, somamos e acrescentamos o viés.

```
z = 0,8·1 + 0,5·(−2) + 0,4 = 0,8 − 1,0 + 0,4 = 0,2
```

(b) O segundo passo é aplicar a ativação ao valor z = 0,2. Cada função recebe o mesmo número e devolve uma saída diferente:

Ativação	Conta	Saída
Degrau	z = 0,2 ≥ 0, então dispara	1
Sigmoide	$1 / (1 + e^{-0,2}) = 1 / (1 + 0,819)$	0,550
Tanh	tanh(0,2)	0,197
ReLU	máx(0; 0,2)	0,2

(c) Com b = −0,4, a soma ponderada das entradas continua valendo 0,8 − 1,0 = −0,2, mas agora ela é somada a −0,4:

```
z = −0,2 + (−0,4) = −0,6
```

O valor de z ficou negativo e o degrau passa de 1 para **0**. Também mudam a sigmoide (0,354), a tanh (−0,537) e a ReLU (0). O limiar de disparo é −b: com b = 0,4 ele valia −0,4 (a soma −0,2 o superava); com b = −0,4 ele passa a +0,4, que a soma −0,2 não alcança.

Ideia-chave. o viés desloca o limiar do neurônio. A mesma soma ponderada pode disparar ou não conforme o viés.

Exercício 2 — Porta NAND

A porta NAND vale 1 em todas as entradas binárias, exceto (1; 1), em que vale 0. (a) Encontre w_1, w_2 e b para que um neurônio com ativação degrau reproduza a NAND. (b) Verifique as quatro linhas da tabela calculando z e y. (c) Descreva a reta de fronteira e diga de que lado ficam os pontos de saída 1.

Resolução.

(a) Em vez de chutar valores, traduza a tabela em desigualdades. O degrau dispara quando $z \geq 0$, e $z = w_1x_1 + w_2x_2 + b$:

Entrada	Saída desejada	Condição sobre z
(0, 0)	1	$b \geq 0$
(0, 1)	1	$w_2 + b \geq 0$
(1, 0)	1	$w_1 + b \geq 0$
(1, 1)	0	$w_1 + w_2 + b < 0$

A primeira condição manda b ser positivo. Para que a soma dos pesos derrube o resultado no caso (1, 1), os pesos precisam ser negativos. Escolhendo **$b = 1{,}5$ e $w = (-1; -1)$** todas as condições se cumprem (qualquer outra escolha que as cumpra também serve).

(b) Conferência com $z = -x_1 - x_2 + 1{,}5$:

x_1	x_2	z	y	NAND
0	0	1,5	1	1
0	1	0,5	1	1
1	0	0,5	1	1
1	1	−0,5	0	0

(c) A fronteira é onde $z = 0$, isto é, a reta $x_1 + x_2 = 1{,}5$. Os pontos (0, 0), (0, 1) e (1, 0) ficam do lado da origem ($z > 0$, saída 1); o ponto (1, 1) fica do outro lado ($z < 0$, saída 0). A NAND é a imagem espelhada da AND da Seção 3: os sinais dos pesos e do viés se invertem.

Ideia-chave. uma porta lógica vira um sistema de desigualdades sobre pesos e viés; a reta $w \cdot x + b = 0$ separa as saídas 1 das saídas 0.

Exercício 3 — Camadas sem ativação

Uma rede tem uma entrada x, uma camada oculta com h = 2x − 1 e uma saída y = −3h + 4. (a) Calcule y para x = 2 passando por h. (b) Escreva y diretamente como função de x. (c) Troque a camada oculta por h = ReLU(2x − 1) e calcule y para x = 0; compare com a fórmula de (b). (d) O que isso diz sobre empilhar camadas sem ativação?

Resolução.

(a) Primeiro a camada oculta, depois a saída:

```
h = 2·2 − 1 = 3          y = −3·3 + 4 = −5
```

(b) Substitua h na segunda equação e simplifique:

```
y = −3(2x − 1) + 4 = −6x + 3 + 4 = −6x + 7
```

Conferência: para x = 2, −6·2 + 7 = −5, o mesmo valor de (a). As duas camadas se reduziram a uma única reta.

(c) Agora a camada oculta tem a ReLU. Para x = 0:

```
h = ReLU(2·0 − 1) = ReLU(−1) = 0          y = −3·0 + 4 = 4
```

A fórmula de (b) daria −6·0 + 7 = 7. Os valores **diferem**: com a ReLU a rede deixou de ser equivalente a uma reta, porque a camada oculta "cortou" o valor negativo.

(d) Camadas sem ativação não acrescentam poder: a composição de funções lineares é linear, e a rede inteira colapsa em uma única camada, não importa quantas existam. É a ativação não linear entre as camadas que permite representar o que uma reta não representa, como o XOR.

Ideia-chave. sem não linearidade, profundidade não acrescenta nada.

Exercício 4 — Saturação

Use σ'(z) = σ(z) · (1 − σ(z)). (a) Calcule σ'(0) e σ'(5), sabendo que σ(5) = 0,9933. (b) Em uma rede com 10 camadas ocultas sigmoides, qual é o valor máximo do produto das 10 derivadas? (c) Explique por que a ReLU atenua esse problema e que risco novo ela traz.

Resolução.

(a) Em z = 0 a sigmoide vale 0,5; em z = 5 vale 0,9933:

```
σ'(0) = 0,5 · (1 − 0,5) = 0,25
σ'(5) = 0,9933 · (1 − 0,9933) = 0,9933 · 0,0067 ≈ 0,0066
```

Em z = 5 a sigmoide está saturada: a derivada é cerca de 38 vezes menor que no centro.

(b) A derivada da sigmoide nunca passa de 0,25 (o valor em z = 0). O gradiente que chega à primeira camada é um produto de 10 derivadas, então o máximo possível é:

$$0{,}25^{10} \approx 9{,}5 \times 10^{-7}$$

Menos de um milionésimo: na prática o gradiente chega nulo às primeiras camadas e os pesos delas quase não mudam. É o **gradiente que desaparece**.

(c) A ReLU tem derivada 1 para z > 0, então o produto de derivadas não encolhe nos neurônios ativos. O risco novo é a **ReLU morta**: se z fica sempre negativo, a derivada é 0 e o neurônio deixa de aprender. A Leaky ReLU mitiga isso mantendo uma pequena inclinação para z negativo.

Ideia-chave. a derivada da ativação multiplica-se camada a camada; derivadas pequenas, em cadeia, apagam o gradiente.

> **Exercício 5 — Softmax e escolha da ativação de saída**
>
> Uma rede produz as pontuações z = (1; 2; 3) na camada de saída. (a) Calcule a softmax. (b) Se somarmos 10 a cada pontuação, a softmax muda? Por quê? (c) Escolha a ativação da camada de saída para: prever o preço de um imóvel; decidir se um e-mail é spam; classificar uma flor em uma de 3 espécies; marcar as etiquetas de uma foto, que pode ter várias ao mesmo tempo.

Resolução.

(a) Calcule o exponencial de cada pontuação, some e divida:

> $e^1 = 2{,}718$ $e^2 = 7{,}389$ $e^3 = 20{,}086$ soma = 30,193
> softmax = (2,718; 7,389; 20,086) / 30,193 = (0,090; 0,245; 0,665)

A maior pontuação recebe a maior probabilidade e as três somam 1.

(b) Não muda. Somar 10 a todas as pontuações multiplica cada exponencial por e^{10}; esse fator aparece no numerador e no denominador e se cancela. Por isso as implementações subtraem o maior valor antes de calcular: evitam números gigantes sem alterar o resultado.

(c) A ativação de saída segue o tipo de resposta que se quer:

Situação	Ativação de saída	Motivo
Preço de um imóvel	identidade (linear)	é regressão: a saída é um valor real qualquer
E-mail é spam ou não	sigmoide (1 neurônio)	saída em (0; 1), lida como probabilidade de spam
Uma entre 3 espécies	softmax (3 neurônios)	classes excludentes: probabilidades que somam 1
Várias etiquetas na mesma foto	sigmoide em cada saída	cada etiqueta é decidida de forma independente

Ideia-chave. a softmax só enxerga diferenças entre as pontuações; a escolha da ativação de saída depende de a resposta ser um valor real, uma probabilidade ou uma classe entre várias.

10. Exercícios de fixação

Resolva no caderno, com cálculo à mão onde indicado. Cada exercício é parecido com um dos resolvidos na Seção 9. O gabarito comentado está em arquivo separado.

Exercício 6 — Calcular a saída de um neurônio

Um neurônio tem entradas x = (2; 3; −1), pesos w = (0,5; −0,4; 0,9) e viés b = 0,5. (a) Calcule z. (b) Calcule a saída com as ativações degrau, sigmoide, tanh, ReLU e Leaky ReLU (α = 0,01). (c) Qual é o menor valor de b para o qual o neurônio de degrau dispara, mantendo as mesmas entradas e pesos?

Exercício 7 — Porta NOR

A porta NOR vale 1 apenas quando as duas entradas binárias são 0 e vale 0 nos demais casos. (a) Escreva as desigualdades que w_1, w_2 e b devem cumprir e encontre valores para um neurônio com ativação degrau. (b) Verifique as quatro linhas da tabela calculando z e y. (c) Descreva a reta de fronteira e diga de que lado fica o único ponto de saída 1.

Exercício 8 — Camadas sem ativação

Uma rede tem uma entrada x, uma camada oculta com $h = 3x + 2$ e uma saída $y = 0{,}5h - 4$. (a) Calcule y para $x = 4$ passando por h. (b) Escreva y diretamente como função de x. (c) Troque a camada oculta por $h = \text{ReLU}(3x + 2)$ e calcule y para $x = -2$; compare com a fórmula de (b). (d) O que esse resultado mostra sobre o papel da função de ativação?

Exercício 9 — Saturação da tanh

Use $\tanh'(z) = 1 - \tanh^2(z)$. (a) Calcule $\tanh'(0)$ e $\tanh'(3)$, sabendo que $\tanh(3) = 0{,}9951$. (b) Compare o valor máximo do produto das derivadas em uma rede de 6 camadas ocultas sigmoides com o de uma rede de 6 camadas ocultas tanh. (c) Apesar do resultado de (b), explique por que a tanh ainda pode sofrer com gradiente que desaparece.

Exercício 10 — Softmax e escolha da ativação

Uma rede produz as pontuações $z = (3; 1; 0)$ na camada de saída. (a) Calcule a softmax. (b) Multiplique todas as pontuações por 2 e calcule a softmax de novo: ela muda? A distribuição fica mais ou menos concentrada? Compare com o efeito de somar uma constante (Exercício 5). (c) Escolha a ativação adequada para: a temperatura prevista para amanhã; aprovado ou reprovado; reconhecer um dígito de 0 a 9 em uma imagem; atribuir a um artigo várias etiquetas ao mesmo tempo; as camadas ocultas de uma rede profunda (ponto de partida).